# Backend (.NET Core) Developer Training Plan

# Training Overview

This learning path is designed to build strong fundamentals in backend development using **Microsoft .NET Core with C#**, secure identity management, multitenant architecture, database development, and deployment practices. The trainee will also build a **HRMS POC Application** as hands-on project work.

> 🔴 **Important items in the roadmap are marked in red.**

# Phase 1: Fundamentals of Git (Practical Required)

## Topics

- What is Git and GitHub
- Local vs Remote repositories
- Basic Git commands (init, clone, commit, pull, push)
- Branching strategy and types of branches
- Merge, conflict handling, rebase, reset, revert
- Cherry-pick

**Reference:**

▸ https://www.youtube.com/watch?v=CvUiKWv2-C0

# Phase 2: C# Language & SOLID Principles

## 2.1 Introduction to C#

- What is .NET, CLR, FCL
- Compilation flow in C#
- Debugging in Visual Studio
- Data types, variables, coding standards

## 2.2 Classes and OOP Concepts

- Types of classes
- Constructors and object creation
- Reference types
- Access modifiers
- Static members & Singleton
- Abstract classes and Interfaces

## 2.3 Types & Assemblies

- Value vs Reference types
- Structs, Enums, Records
- Immutability
- Nullable types
- Assemblies & Namespaces

## 2.4 Members: Methods, Events & Properties

- Methods and types
- Properties, fields
- Delegates and events
- Extension methods

## 2.5 Flow Control

- Conditions
- Loops
- Jump statements

## 2.6 Exception Handling

- Types of exceptions
- Try/Catch/Finally
- Custom exception handling
- `using` keyword

## 2.7 Collections & Generics

- Collection types
- Generic programming

## 2.8 SOLID Principles

**Reference:**

- https://dotnettutorials.net/
- https://www.c-sharpcorner.com/
- YouTube: C# Tutorials

# Phase 3: SQL & PostgreSQL Basics

## Topics

- DDL, DML, DCL, TCL
- ACID Properties
- Filtering, Sorting, Grouping & Paging
- Joins, Subqueries, Apply
- Views & Indexing
- Temp tables, CTE, Case/When
- Stored procedures, functions, triggers
- Cursors, sequences
- SQL profiling and execution plans

**Reference:**

▸ https://www.sqlservertutorial.net/

# Phase 4: Data Access & ORM

## 4.1 ADO.NET

- Connections, commands, data readers
- Connected vs disconnected architecture
- Calling stored procedures

### 4.2 LINQ

- Query syntax & method syntax
- Lambda expressions
- Deferred vs Immediate execution

### 4.3 Entity Framework Core

- What is EF Core? Pros & cons
- Code-First & Database-First approaches
- Migrations and seeding
- Lazy vs eager loading
- Async queries
- Calling stored procedures

### 4.4 Dapper

- Query, Insert, Update using Dapper
- Stored procedures in Dapper
- Dynamic parameter
- Bulk operations

**Reference:**

▸ https://www.learndapper.com/

## Phase 5: ASP.NET MVC & Web API

## 5.1 MVC(Overview Only)

- MVC architecture and lifecycle
- Routing basics
- Controllers & action results
- Filters
- Views & partial views (conceptual only)

**Notice:**

- MVC is covered **only to understand architectural concepts**
- **No MVC-based projects or practical implementation required**
- All hands-on work must be done using **ASP.NET Core Web API**

## 5.2 ASP.NET Web API

- REST vs SOAP
- HTTP verbs, response formats
- API versioning
- Authentication & Authorization (Role-based, Token based, Policy based)
- Token-based security
- OpenID, OAuth
- Caching & localization
- API health check monitoring
- Swagger, Postman testing

### 5.3 .NET MAUI (Basic Usage for Project)

- Introduction to .NET MAUI (successor of Xamarin)
- Cross-platform architecture (Android, iOS, Windows)
- MAUI project structure overview (Pages, App Lifecycle)
- Using MAUI as a **client application** for ASP.NET Core Web API
- Calling REST APIs using `HttpClient`
- Handling request/response (GET, POST – JSON only)
- Basic data binding (overview, not deep MVVM)
- Consuming authentication APIs (login, token storage – basic idea)

## Phase 6: .NET Core

Topics:

- .NET Framework vs .NET Core
- Environment configuration
- Middleware pipeline
- DI Lifecycle: Singleton, Scoped, Transient
- Identity Core authentication
- Asynchronous programming (`async/await`)
- Exception handling & global error handling
- Logging (Serilog)
- Deployment on IIS

**Reference:**

- https://www.youtube.com/watch?v=4IgC2Q5-yDE

## Phase 7: Security

Topics:

- SQL Injection defense
- Anti-forgery tokens
- DoS & DDoS awareness

## Phase 8: Design Patterns

Topics:

- Singleton
- Repository
- Factory & Abstract Factory
- Mediator
- CQRS
- Other common enterprise design patterns

## Phase 9: Latest Enhancements

- C# 12 / 14 features

- .NET 8 / .NET 10 updates

# Phase 10: Azure DevOps & Cloud

- CI/CD pipeline creation
- Repo/Branch strategy
- Deployment automation

**Reference:**
▸ https://learn.microsoft.com/en-us/azure/devops/repos/git/gitquickstart

# HRMS Proof of Concept (POC) Application Development

The goal of this POC is to develop a simple Human Resource Management System where organizations can register and manage employees. The application will demonstrate essential features such as authentication, role-based access, CRUD operations, and API documentation.

---

## Core Features

### 1. User Registration & Authentication

- Implement authentication using **ASP.NET Core Identity Framework**.
- Support secure **user registration and login**.
- Password hashing, validations, and secure JWT token handling.

### 2. Organization Management

- Allow new users to create an **Organization** after registration.
- Required organization details:
    - Name
    - Address

### 3. Role-Based Access

- system roles:
    - **Admin**
    - **HR**

#### Role Permissions:

- **Admin**
    - Can add or remove HR users.
    - Can view all employees, including HR created under the organization.
- **HR**
    - Can create, edit, and delete **only the employees created by themselves**.

### 4. Employee Management

- HR can manage employee records with the following fields:
    - First Name
    - Last Name
    - Email
    - Phone Number
    - Address

---

## Technical Requirements

- **Backend Framework:** ASP.NET Core Web API
- **Authentication:** ASP.NET Core Identity Framework
- **Database:** PostgreSQL
- **ORM Options:**
    - **Dapper** or **Entity Framework Core** (use one or both for demonstration)
- **API Documentation:** Swagger
- **Validation:**
    - Use FluentValidation or Data Annotations for input validation
- **Repository Pattern & Service Layer**
    - Use clean architecture principles for separation of concerns

---

## Source Control & Deployment

- Use **Git** for version control.
- Push all source code to a dedicated **Git repository** (GitHub).